

UTN
TECNICATURA UNIVERSITARIA EN
PROGRAMACIÓN

TRABAJO INTEGRADOR-FOOD STORE

Integrantes:

- MARTÍN, Tomás
- MEDINA, Octavio
- GRIS, Emir
- RAMPONE, Brian

Fecha de entrega: 22/06/26

ÍNDICE:

1. Introducción
2. Marco Teórico
 - 2.1 Java
 - 2.2 Programación Orientada a Objetos
 - 2.3 Bases de Datos Relacionales
 - 2.4 PostgreSQL
3. Decisiones Técnicas y Arquitectura
 - 3.1 Arquitectura General
 - 3.2 Organización de Paquetes
 - 3.3 Entidades del Sistema
 - 3.4 Relaciones entre Clases
4. Dificultades y Soluciones
5. Conclusiones
6. Bibliografía y Webgrafía
7. Repositorio y Video Explicativo

Introducción:

El presente trabajo consiste en el desarrollo de una aplicación denominada FoodStore, realizada en lenguaje Java utilizando conceptos de Programación Orientada a Objetos y persistencia de datos mediante una base de datos relacional PostgreSQL.

El sistema permite gestionar usuarios, productos, categorías y pedidos, implementando una estructura organizada en capas para separar las responsabilidades del acceso a datos, la lógica de negocio y las entidades del dominio.

Durante el desarrollo se aplicaron conceptos fundamentales estudiados en la materia Programación II, tales como herencia, polimorfismo, interfaces, composición, colecciones, excepciones personalizadas y conexión a bases de datos.

El objetivo principal fue construir una aplicación funcional que integrara conocimientos de programación orientada a objetos junto con el manejo persistente de información mediante SQL.

MARCO TEÓRICO

2.1 Java

Java es un lenguaje de programación orientado a objetos ampliamente utilizado para el desarrollo de aplicaciones empresariales, de escritorio y web.

Entre sus principales características se destacan:

- Portabilidad entre sistemas operativos.
- Reutilización de código.
- Gestión automática de memoria.
- Amplio ecosistema de bibliotecas.

En este proyecto Java fue utilizado para implementar toda la lógica del negocio y la comunicación con la base de datos.

2.2 Programación Orientada a Objetos

La Programación Orientada a Objetos (POO) es un paradigma basado en la creación de objetos que representan entidades del mundo real.

Los conceptos aplicados en el proyecto incluyen:

Herencia

Permite reutilizar atributos y métodos comunes mediante una clase base.

Ejemplo:

- Base
- Usuario
- Producto
- Categoría
- Pedido
- DetallePedido

Todas las entidades heredan de la clase Base.

Polimorfismo

Permite que diferentes objetos respondan a un mismo comportamiento.

En el proyecto se aplica mediante la interfaz Calculable implementada por la clase Pedido.

Encapsulamiento

Los atributos de las clases se encuentran protegidos mediante modificadores de acceso y se manipulan a través de métodos getters y setters.

Composición

La clase Pedido contiene una colección de objetos DetallePedido, formando una relación de composición.

2.3 Bases de Datos Relacionales

Las bases de datos relacionales organizan la información en tablas relacionadas mediante claves primarias y foráneas.

Las principales ventajas son:

- Integridad de datos.

- Consistencia.
- Facilidad para realizar consultas complejas.
- Persistencia de la información.

En FoodStore se utilizan tablas para almacenar usuarios, productos, categorías, pedidos y detalles de pedidos.

2.4 PostgreSQL

PostgreSQL es un sistema de gestión de bases de datos relacional de código abierto.

Fue elegido debido a:

- Su estabilidad.
- Compatibilidad con Java.
- Soporte para transacciones.
- Facilidad de integración mediante JDBC.

La base de datos se encuentra alojada en la nube utilizando el servicio Neon.

3. DECISIONES TÉCNICAS Y ARQUITECTURA

3.1 Arquitectura General

Se decidió utilizar una arquitectura en capas para lograr una correcta separación de responsabilidades.

La estructura general se compone de:

- Config
- Entities
- Repository
- Service
- Main
- Exception
- Enums

- Test Connection

Esta organización facilita el mantenimiento, la escalabilidad y la reutilización del código.

- Estructura de paquetes del proyecto.

3.2 Organización de Paquetes

Config

Contiene la configuración de la conexión con PostgreSQL.

Clase principal:

- DatabaseConnection

Responsabilidad:

- Abrir conexiones.
- Gestionar credenciales.
- Centralizar el acceso a la base de datos.

Entities

Representan las entidades del negocio:

- Usuario
- Producto
- Categoría
- Pedido
- DetallePedido

Repository

Implementa el acceso a los datos mediante consultas SQL.

Su función es actuar como intermediario entre Java y PostgreSQL.

Las sentencias que interactúan con la base de datos utilizan Prepared Statements (o consultas parametrizadas) para prevenir ataques de Inyección SQL.

Service

Contiene las reglas de negocio y validaciones del sistema.

Los servicios coordinan las operaciones entre entidades y repositorios.

Test Connection

Verificación de conexión entre el sistema y la base de datos.

3.3 Entidades del Sistema

Usuario

Representa a los usuarios que operan dentro del sistema.

Atributos principales:

- Nombre
- Email
- Rol

Producto

Representa los productos disponibles para la venta.

Atributos:

- Nombre
- Precio
- Stock
- Categoría

Categoria

Permite clasificar los productos.

Pedido

Representa una compra realizada por un usuario.

Incluye:

- Usuario asociado
- Estado
- Forma de pago
- Lista de detalles

DetallePedido

Representa cada producto agregado dentro de un pedido.

Contiene:

- Producto
- Cantidad
- Precio

3.4 Relaciones entre Clases

Herencia

Todas las entidades heredan de la clase Base.

Esta clase concentra atributos comunes como:

- id
- eliminado
- fecha de creación

Asociación

Existe una asociación entre:

- Pedido y Usuario
- Producto y Categoría

Composición

Pedido contiene una lista de DetallePedido.

Un detalle no tiene sentido fuera de su pedido correspondiente.

Dependencia

Los Services dependen de los Repository para realizar operaciones sobre la base de datos.

Interfaces

La interfaz Calculable define comportamientos relacionados con cálculos monetarios.

La clase Pedido implementa esta interfaz para calcular el total de la compra.

4. DIFICULTADES Y SOLUCIONES

Durante el desarrollo surgieron diversos desafíos técnicos.

La principal dificultad estuvo relacionada con la integración entre Java y PostgreSQL.

Inicialmente se presentaron problemas de conexión debido a errores en la configuración del driver JDBC y en las credenciales de acceso a la base de datos alojada en Neon.

Además, fue necesario comprender el funcionamiento de las consultas SQL, el manejo de conexiones y la conversión de datos entre objetos Java y registros de la base de datos.

Para resolver estos inconvenientes se investigó la documentación oficial de PostgreSQL y JDBC, realizando múltiples pruebas de conexión hasta lograr una comunicación estable entre la aplicación y la base de datos.

También se presentaron errores relacionados con validaciones de negocio, como cantidades inválidas o falta de stock disponible.

Estos problemas fueron resueltos mediante la implementación de excepciones personalizadas utilizando la clase ValidacionException.

Gracias a estas soluciones se logró desarrollar una aplicación estable, organizada y funcional.

5. CONCLUSIONES

La realización de este proyecto permitió integrar conocimientos de Programación Orientada a Objetos, bases de datos relacionales y desarrollo de aplicaciones en Java.

Durante el trabajo se aplicaron conceptos fundamentales como herencia, polimorfismo, encapsulamiento, interfaces, composición, persistencia, colecciones y manejo de excepciones.

Además, se adquirió experiencia práctica en la conexión de aplicaciones Java con PostgreSQL utilizando JDBC.

El resultado final es una aplicación capaz de gestionar productos, usuarios y pedidos de manera eficiente, manteniendo la información almacenada de forma persistente en una base de datos relacional.

6. BIBLIOGRAFÍA Y WEBGRAFÍA

Oracle. Java Documentation.

<https://docs.oracle.com/javase/>

PostgreSQL Official Documentation.

<https://www.postgresql.org/docs/>

JDBC Documentation.

<https://docs.oracle.com/javase/tutorial/jdbc/>

Neon PostgreSQL Documentation.

<https://neon.com/docs>

GitHub.

<https://github.com/>

Documentación utilizada durante el desarrollo y resolución de problemas de conexión.

7. REPOSITORIO Y VIDEO EXPLICATIVO

Repositorio del proyecto:

<https://github.com/ocmedina/FoodStore-UTN>

Video explicativo:

https://drive.google.com/drive/folders/1HrJBHdHY2tXS-Q3_T7wYYrIDtQeyo7Bw?usp=sharing

ANEXO – CAPTURAS DEL SISTEMA

- Menú principal.



```
Run Main x
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\id...
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: |
```

- Conexión exitosa con PostgreSQL.



```
Run TestConnection x
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\id...
Conectando a Neon PostgreSQL...
¡CONECTADO! (3370 ms)
Versión: PostgreSQL 18.4 (48c2093) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Process finished with exit code 0
```

- Alta o listado de productos.



```
Run Main x
--- GESTIÓN DE PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
Producto [ID=1] - Vacío | Precio: $23000.0 | Stock: 0 | Cat: CARNES
Producto [ID=2] - Costilla | Precio: $12000.0 | Stock: 6 | Cat: CARNES
```

- Gestión de pedidos.



```
Run Main x
--- GESTIÓN DE PEDIDOS ---
1. Listar
2. Crear Pedido
3. Actualizar Estado
4. Eliminar
0. Volver
Seleccione: 1
Pedido [ID=5] - 2026-06-22 | Usuario: Tomás | Estado: CONFIRMADO | Pago: TRANSFERENCIA | Total: $69000.0
Pedido [ID=6] - 2026-06-22 | Usuario: Tomás | Estado: PENDIENTE | Pago: EFECTIVO | Total: $48000.0
```

- Estructura de paquetes del proyecto.

